Swarma: платформа для роёв автономных агентов

Рантайм на машинах владельца, протокол совместной разработки и цепочка идентичностей

Алексей Григорьев, Иван Опарин
Relux Works
github.com/relux-works

Октябрь 2026, четвёртая редакция, версия 4.0

Аннотация. Swarma — платформа для роёв автономных агентов: рантайм на машинах одного владельца, протокол совместной разработки для независимых владельцев и предлагаемая цепочка идентичностей. Общая задача слоёв — отделить способность модели предложить действие от полномочия его совершить. Swarm Runtime задаёт границы пользователей ОС, хранение ключей вне агента, делегирование прав и управляемую доставку в сессии. Swarm Protocol связывает работу над единой авторитетной средой с журналом, кворумом, независимым ревью и атомарным продвижением точно проверенного кандидата. Swarm Chain предлагается как общий источник имён, связей ключей и аттестаций, смысл которых вычисляют индексаторы. Мы различаем работающие компоненты, решения, спецификации, предложения и открытые вопросы. Для ограниченного фрагмента интеграции воспроизведён исчерпывающий перебор: 1090 состояний без нарушений; отключение каждого из четырёх правил даёт контрпример. Это не доказательство всей платформы. Полезность роя, качество ревью, развёртывание рантайма и переносимая репутация требуют отдельных проверок. Вклад работы — композиция границ доверия с явными допущениями и планом экспериментов, способных опровергнуть её пользу.

1. Введение

Агент разработки всё реже ограничивается одним запросом. Он читает чужие документы, запускает инструменты и меняет реальные системы. Среда часто знает только запустившего процесс пользователя: его права достаются всем агентам. Инъекция в промпт использует эти права руками самого агента. Просьба не читать секрет слабее границы, за которой процесс его прочитать не может.

Переход от одного агента к рою добавляет другую проблему: как объединить ёмкость независимых владельцев на общем проекте. Среда разработки может быть дорогой или невоспроизводимой: особый тулчейн, лицензии, оборудование, данные и ключи подписи привязаны к машине. Параллельные агенты одного владельца расходуют одну квоту провайдера. Чужой владелец не должен передавать проекту свой логин, а проект не должен отдавать ему административный доступ.

Крупные координируемые рои доступны прежде всего вендорам моделей и организациям с собственными ресурсами; открытые проекты по-прежнему координируются через людей. Возможный выигрыш от автономной координации — гипотеза. Проверять её нужно на одинаковых внешних задачах, учитывая цену координации.

В формальной математике результат агента проверяет ядро доказательств. У обычного программного проекта такого оракула принятия нет: успешная сборка не доказывает соответствия намерению. Поэтому принятие приходится конструировать из контракта задачи, проверок, независимого суждения и правил интеграции. Например, исправление может пройти тесты на старой базе, но стать неверным после изменения соседнего интерфейса.

Четвёртая редакция расширяет предмет с распределённого протокола до платформы из трёх слоёв. Мы описываем, где возникает авторитет, кто вправе менять состояние и к каким версиям относятся свидетельства. Где эта работа и спецификация модуля расходятся, права спецификация. Неопубликованные форматы здесь не достраиваются. Далее пометки различают пять состояний: ⟦работает сегодня⟧, ⟦решено⟧, ⟦спецификация⟧, ⟦предложено⟧, ⟦открыто⟧. Решение не означает реализации, а проверка фрагмента не переносится на всю систему.

2. Модель: три слоя и участники

Swarm Runtime, слой 2, организует агентов на машинах одного владельца: учётки ОС, сеть, ключи, гранты, сессии и процессы. Здесь уже работают отдельные компоненты; целевая граница отдельного пользователя для каждого автономного агента ещё строится. **Swarm Protocol, слой 1,** описывает совместную разработку независимых владельцев над единой средой исполнения: кор-ноды ведут журнал, рабочие ноды предлагают изменения, принятие проходит независимое ревью. Распределённая система пока не реализована. **Swarm Chain, слой 0,** — замысел общего состояния для денег, имён, иерархий ключей и социальных записей. Поэтому строим сверху вниз: работающему рантайму не требуется готовая цепочка. ⟦предложено⟧

Владелец отвечает за свои машины, агентов и источник ёмкости модели. Оператор совершает привилегированные действия; это роль человека, а не ещё один агент с широким промптом. Агент — длительно живущий участник с личностью и ограниченными полномочиями. Контрибьютор отдаёт проекту ёмкость своего агентского цикла; донор — владелец такой ёмкости. Проект задаёт цель, среду и правила принятия. Совмещение ролей не даёт дополнительных прав.

Граница рантайма рассчитана на агента, который может быть захвачен инъекцией. Ядро ОС, root, установка платформы и доверенные посредники входят в основание доверия. Граница протокола рассчитана на недоверенные рабочие ноды и, на соответствующем уровне, неисправные кор-ноды; среда проекта доверена в отношении целостности. Swarm Chain рассматривает открытую сеть, в которой подпись подтверждает происхождение записи, но не честность участника. Эти модели противника различны: проверка подписи не доказывает соблюдения сетевой политики, а кворум не исправляет ложный отчёт скомпрометированной среды.

У слоёв есть общий сценарий. Внешний агент проверяет кандидата проекта, не получая его секретов; проект проверяет полномочия и происхождение вердикта. Публикация исхода в общем пространстве могла бы позволить агенту предъявить историю другому проекту. Первая часть относится к устройству протокола, последняя —

к предложению переносимой репутации; единого работающего пути через все три слоя пока нет. ⟦предложено⟧

3. Сквозные принципы

Ключ задаёт криптографическую личность, но не лежит у агента. В рантайме агент просит ключника подписать сообщение; другим владельцам предъявляет публичный ключ и удостоверения. Для Swarm Chain предлагается иерархия с внешним родительским ключом. Компрометация дома агента не должна раскрыть мастер-секрет. Это не запрет злоупотребления разрешённой подписью: полномочия проверяются отдельно. До появления ключника существуют явно названные исключения, описанные в разделе «Swarm Runtime». ⟦предложено⟧

Полномочия только сужаются. Грант — подписанное разрешение на действия над конкретными целями. Делегирование не расширяет действия, область и срок родителя. В протоколе цель содержит полномочие, оно — задачу, задача — ревизию в разрешённой области записи. Например, разрешение менять реализацию функции не даёт продьюсеру права переписать критерии её принятия. Для цепочки предлагаются записи связей ключей; сами по себе они не выдают грант в чужом рантайме. ⟦предложено⟧

Журнал авторитетен, сессия модели — кэш. Назначение, результат, передача и основание принятия должны переживать потерю контекста. Преемник восстанавливает обязательства по записям их владельцев, а не по уверенной фразе старой сессии. У доски, хоста сессий и кипера свои области состояния; принцип не требует одного глобального журнала всех событий. ⟦предложено⟧

Суждение моделей остаётся вне консенсуса. Валидатор проверяет подписи, версии, вложенность областей и допустимость перехода. Он не голосует за красоту кода. Аналогично мост может дополнительно сузить допуск модельной проверкой, но модель не создаёт отсутствующее право. В цепочке предлагается отделить проверку формы социальных записей от детерминированного вычисления их смысла индексаторами. ⟦предложено⟧

Одобрение адресовано точному предмету. В протоколе адрес аттестации состоит из версии контракта, кандидата, политики проверки и окружения. Другое состояние требует новых оснований принятия. Для памяти предлагается тот же подход к оценкам и ссылкам на источники; для переносимых аттестаций точный формат ещё открыт. Слова «это уже проверяли» недостаточно, если проверяли другую базу или другой контракт. ⟦предложено⟧

Непроверяемое не становится разрешённым. Неизвестный исход не равен успеху, нечитаемый реестр отзыва — отсутствию отзыва. В рантайме неготовая граница не называется принудительной; в протоколе отсутствие независимого ревьювера ведёт к ожиданию или отдельной авторизации изменения условий. ⟦предложено⟧

Решение человека удостоверяется его ключом. Принят путь подписи точного действия оператором с телефона после показа содержимого и биометрии; запасной путь — отдельная операторская учётка с подтверждением в терминале. Ключ телефона хранится в защищённом чипе, а произвольная агентская сессия не заменяет присутствия человека. Регистрация и восстановление ключа требуют дальнейшего уточнения. ⟦решено⟧

4. Swarm Runtime: рой на машинах одного владельца

Граница учётки и сети. Каждому автономному агенту назначается отдельный пользователь ОС: собственный дом и процессы. При корректных правах, закрытой сети и доверенном root другой агент не должен читать его файлы, память или писать ему в терминал. Выбран короткий путь через существующую границу ядра без обязательных контейнеров, образов и виртуальных машин. Самостоятельный репозиторий нужен и для кода: связанный `git worktree` между учётками оставлял бы общие настройки и хуки. Внутри одной учётки рабочие деревья допустимы. ⟦решено⟧

Учётка сама по себе не ограничивает сеть. В целевом режиме файрвол по UID направляет локального агента только к слушателю доверенного прокси его профиля, а профиль выбирает политика машины. Unix-сокеты допускают собеседника по данным ядра. Кооперативная настройка прокси уже существует, но не является принуждением. Квалификация должна проверять IPv4, IPv6, DNS, loopback, открытые соединения и взаимодействие правил. CONNECT ограничивает имя, порт и адрес соединения, но не содержимое TLS-туннеля: разрешённый адресат остаётся каналом выноса. ⟦спецификация⟧

Менеджер пользователей (`swarma-user-manager`) создаёт и выводит учётки; диспетчер (`swarma-dispatcher`) допускает заявки, учитывает квоты и вызывает узкий запускатель. Поколение учётки отличает новое использование UID от старого: повторный номер не должен вернуть чужие права. При неизвестном исходе создания или запуска диспетчер сверяет тот же эффект, а не создаёт новый. Вывод идёт вперёд после сбоя; дом удаляется, поскольку сканирование файлов не доказывает отсутствия спрятанного токена. Сохранение дома требует отдельного решения оператора. ⟦спецификация⟧

Ключник и гранты. Ключник — отдельный привилегированный сервис с мастер-секретом. Принят стабильный путь личности вида «назначение / продукт / проект / агент», не зависящий от UID и каталога. Собеседник устанавливается по UID от ядра и активному поколению регистрации, а не по имени из запроса. Вывод учётки отключает доступ к ключу, но не равен отзыву постоянной личности. Модуля и спецификации ключника пока нет; иерархическая деривация с аппаратным хранением остаётся предложением. ⟦предложено⟧

Общий грант проверяется у получателя действия: подписи всей цепочки, сужение, сроки, условия и отзыв. Оркестратор просит, а не судит. В спецификации запуска права ребёнка зависят и от диспетчера, и от вызывающего; отзыв оркестратора должен отрезать его потомков. Неизвестность отзыва закрывает допуск. До ключника ключ заявок вызывающего лежит в его доме — исключение, ограниченное его грантом. Брокер учётных данных (`swarma-credential-broker`) ограничивает выдачу токена, но процесс модели способен прочитать выданный ему токен. ⟦спецификация⟧

Сообщения и сессии. Мост принимает подписанные сообщения поверх шифрования носителя; членство в комнате Matrix не выдаёт прав. Провайдер сохраняет событие, ворота проверяют точные байты, подпись, гранты и повтор, затем сообщение попадает в долговечную очередь. Агент получает короткий звонок и вытягивает тело из почтового ящика как данные. Проверка небольшой моделью может только сузить допуск. Исходящие байты подписываются собственным ключом агента и не переписываются мостом. Это целевая архитектура; сегодня используется сам приём звонка и вытягивания. ⟦предложено⟧

Хост сессий (`agent-session-host`) отделяет службу учёта от раннера под учёткой агента. Ввод имеет одного писателя; старый контроллер отсекается эпохой. При перезапуске доставка закрывается до сверки старого исполнения, привязок и неизвестных доставок. Запись в канал ещё не доказывает применения сообщения. Привилегированный старт остаётся у диспетчера; хост сессий запрашивает его по разрешению владельца. Форматы этой связки ещё не опубликованы, поэтому её нельзя считать включённой. Действующий демон сессий внутри доски не обладает автоматически гарантиями будущего разделения. ⟦спецификация⟧

Обязательства и конфигурация. Swarm Keeper — главный цикл узла и предлагаемое ядро обязательств: кто кому что должен и к какому сроку. Передача завершается первым подтверждённым действием нового держателя, а не ответом «принял». Если новый ревьювер согласился, но не начал работу, обязанность остаётся у прежнего держателя и эскалируется. Истечение срока не закрывает обязательство. Журнал одного писателя позволяет восстановить положение после сбоя; наблюдатель может предложить изменение, но не принять его за владельца состояния. ⟦предложено⟧

Swarm Curator владеет профилями, скиллами, MCP и содержимым управляемых домов. Доска не должна создавать второй менеджер профилей. Существуют общие и изолированные логины; выбор хранилища зависит от машины и среды. Для живой сессии решён собственный снимок, поскольку управляемый дом может обновляться. Проверка готовности должна различать известное отсутствие логина и неизвестность; последняя не разрешает автоматически начать новый вход. Снимок окружения не переносит секрет в план. ⟦решено⟧

Удалённая работа и процесс. Удалённому агенту проект выделяет рабочее место под отдельной учёткой. «Модель думает у донора, руки — у нас»: логин провайдера остаётся у донора, вызовы инструментов исполняются у проекта. Рабочее место не имеет исходящей сети; входящее SSH-соединение — другая граница. На стороне донора первый вариант убирает локальные инструменты из проверенной сборки харнеса, но не даёт отдельной границы ОС. Варианты с отдельными учётками или песочницей требуют собственной квалификации. Совместная жизнь SSH-реле, остановки и переподключения ещё открыта. ⟦предложено⟧

Доска задач (`task-board`) связывает продьюсера, ревьювера и Change Request: принятый кандидат интегрируется через compare-and-set, то есть замену ожидаемой головы. Доска остаётся необязательным клиентом платформы. Нынешний цикл работает, а перенос его исполнения в новые модули ещё проектируется. Неагентский исполнитель публикации, передачи кандидата на проверку и интеграции пока не выбран; владение процессом не даёт агенту доступа к хостингу кода. ⟦открыто⟧

Маршрутизатор моделей ранжирует только уже допущенные конфигурации по меткам задачи, измерениям качества и бюджету. Он не запускает процессы, не расширяет допуск и не вызывает модель для выбора. Неизвестное качество не становится разрешением выбрать кандидата; качество решения зависит и от корректности меток. Существуют библиотека и командный интерфейс. ⟦работает сегодня⟧

Память агентов предлагается как ограниченная доказательная память: утверждения, ссылки на источники, обоснования, оценки и ревизии. Она отвечает, какие допущенные свидетельства поддерживают или опровергают утверждение, а не что истинно и кому можно действовать. Пустой неполный поиск означает неизвестность. Первый реестр допускает нечувствительные документы через ревью человеком; сырые журналы не принимаются автоматически. Снимок приходит от-

дельным блоком данных, не инструкциями. Неизменность снимка не продлевает разрешений. Реализации и измеренного выигрыша пока нет. ⟦предложено⟧

Модуль или срез	Состояние
Доска и её демон сессий; профили и дома Swarm Curator; маршрутизатор; кооперативные сетевые профили; звонок и вытягивание	⟦работает сегодня⟧
Менеджер пользователей, диспетчер, брокер, целевой хост сессий и принудительная сеть	⟦спецификация⟧
Ключник, общий мост и гранты; Swarm Keeper; удалённое рабочее место; память	⟦предложено⟧

5. Swarm Protocol: много владельцев, одна среда

Топология. Среда исполнения проекта — единственный логически авторитетный экземпляр основной ветки, санбоксов и проверок. Физически допустимы исполнительные реплики, но голова меняется в одном месте. Среда пассивна: агентские циклы работают на кор-нодах и машинах контрибьюторов. По сети проходят команды, вывод, хэши и подписи. Кор-ноды ведут журнал и проверяют правила; рабочие ноды не доверены. Единство среды экономит её воспроизведение, но не освобождает от проверки её честности. Далее описывается проект протокола, а не действующее распределённое развёртывание. ⟦предложено⟧

Журнал и кворум. Запись связывается хэшем с предыдущей; дерево Меркла позволяет доказать включение под подписанным снимком. Тела адресуются по содержимому, а их доступность подтверждают независимые реплики. Сертификат кворума, QC, означает достаточное число подписей для объявленного уровня доверия. Лидер, упорядочивающий записи, и оркестратор, планирующий задачи, — разные роли. Среда проверяет QC, включение и актуальность разрешения на границе исполнения; хранит высшую принятую версию журнала, чтобы не принять старый снимок. ⟦предложено⟧

На L0 одна кор-нода: QC — одна подпись, кор и среда доверены. На L1 при $n \geq 2$ нужны $t > n/2$ подписей и честный секвенсер. Это препятствует одиночному несанкционированному решению, но единственность истории зависит от секвенсера. На L2 при $n \geq 3f+1$ используются BFT-консенсус и $2f+1$ подписей; допускается до f злонамеренных кор-нод [1, 2]. Большинство само по себе не заменяет консенсус: при трёх участниках злонамеренный может подписать две несовместимые записи, а честные — по одной каждый; обе соберут две подписи. Пересечение кворумов должно содержать честного участника, для чего требуется $2t - n > f$. ⟦предложено⟧

Контракт и полномочия. Контракт задачи фиксирует родительскую цель, область записи, версии входов, ожидаемый результат, критерии принятия, профиль подтверждаемых свойств, риск и бюджеты. План отвечает на другой вопрос: какие инструменты, гранты и лимиты разрешены исполнению. Его digest включён в назначение и сверяется на вызове. Например, изменение реализации библиотеки допускается в своей области, но изменение защищённых проверок требует задачи соответствующего класса. Новые критерии создают новую версию контракта с отдельной авторизацией. ⟦предложено⟧

Валидаторы проверяют допустимость перехода, вложенность полномочий, текущие

версии, независимость ролей, аренду, лимиты и доступность тел. Они не определяют качество реализации. Две задачи с пересекающимися областями записи не исполняются одновременно; область удерживается до интеграции или отмены. Непересекающиеся пути не исключают семантического конфликта: новый тип в одном модуле может сломать использование в другом. Поэтому принятие относится к целому кандидату, а не только к локально изменённым файлам. ⟦предложено⟧

Среда и свидетельства. Назначение открывает доступ только к санбоксу задачи. Копия дерева не является границей: нужны квалифицированные ограничения ОС. Для целевого рантайма применяется пользователь ОС с сетевой границей, описанной выше. Среда публикует инвентарь того, что умеет ограничивать; требование полной границы не удовлетворяется частичной. Команды связываются цепочкой квитанций, receipts, с входным и результирующим состоянием. Квитанция удостоверяет происхождение отчёта среды, а не воспроизводимость: последней нужны сохранённые входы, версии тулчейна, зависимости и существенные внешние ответы. ⟦предложено⟧

Оркестратор по аренде. Оркестратор декомпозирует цель и предлагает назначения в пределах своих полномочий. Аренда задаёт область, эпоху и fencing-токен; просроченная запись отвергается и сохраняется как расхождение. Преемник определяется ротацией. Плановая передача требует покоя: нет незавершённых вызовов, дочерних процессов и операций с внешними эффектами. При принудительном захвате старый держатель отсекается токеном, сессия не передаётся, новый стартует из журнала. Даже планово полученный контекст считается недоверенным. ⟦предложено⟧

От ревизии к кандидату. Продьюсер представляет подписанную ревизию R над базой B. Ранняя обратная связь помогает, но ещё не является одобрением интеграции. Среда строит неизменяемый кандидат I над текущей головой M и фиксирует пару (M, I). Множество допустимых ревьюверов замораживается до появления хэша кандидата; случайность из проверяемой случайной функции или порогового маяка получается после. Выбор вычисляется из задачи, хэша и случайности, поэтому оркестратор не назначает удобного ревьювера после чтения результата [6]. ⟦предложено⟧

Ревьювер оценивает разницу $M..I$ и подписывает одобрение. Проверки инициирует участник, независимый от продьюсера и ревьювера согласно политике; критические наборы повторяет верификатор. Адрес каждой аттестации — четвёрка «контракт, кандидат, политика, окружение». Исходы: «пройдена», «провалена», «не установлена». Сбой или таймаут дают последний исход, который не удовлетворяет профилю принятия. Нет независимого участника — ожидание, эскалация или отдельно авторизованное изменение условий, а не автоматическое ослабление. ⟦предложено⟧

Интеграция. Запись с QC разрешает атомарную замену M на I, если текущая голова всё ещё равна M. Когда голова сдвинулась, кандидат перестраивается, ревью и проверки повторяются. Даже одинаковый patch-id на другой базе не переносит одобрение в базовой гарантии. Например, параллельная задача могла поменять смысл вызываемой функции, не затронув строк патча. Compare-and-set предотвращает применение старого решения к новому состоянию, а повторное ревью проверяет изменившийся смысл. ⟦предложено⟧

Внешние эффекты. Подпись, публикация и другие операции с секретами проекта доступны только именованному исполнителю, не shell санбокса. У операции есть идентификатор и ключ идемпотентности; повтор сохраняет параметры. Если исполнитель упал до записи исхода, исход неизвестен: требуется сверка получателя или

решение человека. Нельзя обещать отсутствие повторного эффекта у получателя, который не поддерживает идемпотентность. Новая эпоха требует актуальной авторизации; уже начатая операция доводится до учёта исхода. Конкретный исполнитель в новой архитектуре рантайма остаётся открытым. ⟦предложено⟧

Любая кор-нода может подписать экстренную остановку. Среда прекращает новые назначения, операции и продвижение головы; возобновление требует QC уровня. Это сознательный обмен живучести на целостность. Федерация проектов строится через подписанный манифест: среда, уровень доверия, ограничения, допустимые провайдеры и политика независимости. Присоединение означает доверие контрибьютора среде и кор-команде проекта; обратное доверие не возникает. ⟦предложено⟧

6. Идентичность, аттестации и репутация

Предлагается общий путь: внешний агент приходит с публичным ключом и цепочкой удостоверений до владельца, рантайм проверяет её по записям Swarm Chain, а история проверенных работ доступна следующим проектам. Ключ служит предъявляемым идентификатором, но личность переживает ротацию рабочих ключей благодаря иерархии. Это соединение трёх слоёв пока является направлением, не решением и не спецификацией. ⟦предложено⟧

Опоры уже описаны раздельно. Ключник рантайма связывает агента со стабильной личностью под корнем владельца. Протокол предусматривает долгоживущий ключ идентичности, удостоверяющий короткоживущие ключи нод, и подписанные исходы задач. Swarm Chain предлагает связи ключей: запрос, принятие и отзыв, с внешним родителем вне повседневных устройств. Новая часть — общее место аттестаций вне журналов отдельных проектов, не принадлежащее одному рантайму. ⟦предложено⟧

После ревью или принятия работы ключ идентичности проекта подписывает аттестацию. Предлагаемый минимальный состав: личность агента; роль продьюсера, ревьювера или верификатора; ссылка на проект; хэш кандидата или ревизии; версии контракта и политики; исход «принято», «отклонено» или «не установлено»; время и подпись выдавшего. Это перечень содержания, а не формат на проводе. В цепочку не должны попадать код, промпты, данные проекта и секреты — только хэши и метаданные. ⟦предложено⟧

Репутацию предлагается вычислять детерминированно индексаторами как функцию над аттестациями. Вес свидетельства зависит от истории выдавшего проекта: новый рантайм не получает доверие одной самоподписанной записью. Другой проект применяет собственную политику допуска и глубины проверки; переносимы свидетельства, а не право работать. По предложенному сценарию новому агенту нужны два-три независимых ревьювера и низкий вес при выборе в ревьюверы, агенту с историей — один ревьювер и больший вес. Это направление адаптивной проверки, не готовые пороги безопасности. ⟦предложено⟧

Например, агент проверил кандидат библиотеки в одном проекте и предъявляет исход другому. Второй проверяет издателя, предмет аттестации, актуальность ключей и применяет свою политику. Хорошая история ревью не означает автоматически хорошей работы продьюсера. Ротация короткоживущего ключа не обнуляет историю личности; записи, подписанные отозванным ключом после отзыва, предлагается исключать из расчёта. Конкретная формула и разделение репутации по ролям ещё не определены. ⟦предложено⟧

Независимость имеет три оси: административный домен, семейство модели и тулчейн. Первая уменьшает риск сговора и множественных личностей, остальные — общих слепых зон. Проект указывает, что подтверждено и что заявлено. Разные строки владельца не доказывают разных владельцев; третья нода, запустившая тесты в той же среде, разделяет роли, но не доказывает честности среды. Для этого нужен верификатор на другой машине. ⟦предложено⟧

В протоколе доказательство идентичности двустороннее: подписанное заявление и отпечаток ключа у внешнего провайдера. Предусмотрены TXT под DNSSEC, well-known по HTTPS и ключи аккаунта на хостинге кода. Внешний запрос недетерминирован, поэтому подтверждается совпавшее свидетельство валидаторов в одном окне, а не самостоятельное сетевое наблюдение каждого как правило консенсуса. Недоступность провайдера позже не отменяет зафиксированного наблюдения; политику переподтверждения задаёт проект. Доказан контроль над доменом или аккаунтом, не честность. ⟦предложено⟧

Создание новых личностей не исключено [5]; ограничения репутационных механизмов не позволяют обещать одновременно пропорциональное вознаграждение и полную устойчивость к множественным личностям [4]. Аттестация фиксирует, что проверка состоялась и чем закончилась, а не правильность суждения. ⟦предложено⟧

Открыты формат и стоимость записи, защита от спама, круг допустимых издателей, отрицательные аттестации, споры и апелляции, затухание и оси репутации, связь с оплатой ревью. Публичность раскрывает отношения участников. Для частных проектов обсуждается публикация обязательства-хэша с выборочным раскрытием, но готового механизма нет. Бесплатный или почти бесплатный текст в цепочке сам по себе антиспам не решает. ⟦открыто⟧

7. Swarm Chain

Swarm Chain — нижний слой общего состояния, а не распределённый диск рантайма. Кода цепочки в исходном срезе нет. В замысле три подслоя: цепочка упорядочивает записи и время; p2p-память держит тяжёлый контент; социальный граф направляет распространение и архивирование. Деньги и имена требуют консенсуса. Для социальных записей узел проверяет форму, подпись и оплату, а смысл детерминированно вычисляют индексаторы. Это принятое направление, не реализация. ⟦решено⟧

P2P-память не гарантирует хранения: контент может исчезнуть, даже если указатель остался в цепочке. Социальный граф делает выбранное участниками содержимое доступнее, но не превращает добровольную раздачу в обязательство вечного архива. Первым транспортом выбран BitTorrent. Принятие записи и её показ различаются: политику показа и категоризации предлагается оставить клиентам и кураторам. Ни подписка, ни оценка популярности не становятся полномочием в проекте. ⟦предложено⟧

Нативный словарь по образцу Memo включает публикации, ответы, отметки, подписки, профили, связи ключей и указатели на контент [11]. Его смысл не должен принадлежать единственному серверу: предлагается каноническое вычисление производного состояния и сравнение хэшей нескольких индексаторов. Точные схемы и лимиты требуют отдельной спецификации. Коды Memo не объявляются кодами Swarm Chain; образец протокола не заменяет собственный контракт. ⟦предложено⟧

Личность — иерархия с внешним родительским ключом, который позволяет удосто-

верять дочерние ключи и отзывать скомпрометированные [12]. Имя указывает на корень иерархии. Для имён предлагаются раздельные пространства людей, агентов и доменов, регистрация через обязательство и раскрытие, платное продление и зависимость цены от длины. Урок Namecoin — уникальности недостаточно, если удержание имён не сдерживает сквоттинг [13, 14]. Конкретная цена и её изменение со спросом открыты. ⟦предложено⟧

Для консенсуса рассматриваются доказательство работы с CPU-ориентированным RandomX, пересчёт сложности ASERT и плавная эмиссия с хвостом [16, 15, 17]; параметры и выбор объединённого майнинга не решены. Требование распределять ресурс без дешёвого преимущества специализированного оборудования не доказывает справедливости экономики и не исключает ботнетов или аренды CPU. Переход от этого замысла к цепочке требует симуляций, словаря записей, спецификации индексатора и прототипа. ⟦открыто⟧

8. Что проверено и как проверить остальное

Исполняемая модель в `paper/model/` проверяет фрагмент «кандидат — аттестация — запись интеграции — продвижение головы». Две задачи позволяют моделировать сдвиг базы; версии контракта и эпохи ограничены двумя, каждая запись доставляется не более двух раз. Проверяются совпадение головы с базой, текущая эпоха, версия контракта и адресация аттестации к кандидату. Повторный запуск приложенной модели дал следующий вывод; числа справа — состояния, исследованные до контрпримера, не размеры полного пространства мутанта.

Правила	Состояния	Результат
Все	1090	Нарушение не найдено
Без `cas`	84	Запись применена дважды
Без `epoch`	82	Устаревшая эпоха продвинула голову
Без `contract`	71	Аттестация ссылается на заменённый контракт
Без `bind_I`	490	Аттестация сделана на иной базе, чем принятый кандидат

Это исчерпывающий перебор только заданных границ. Модель считает среду и валидаторов честными, не моделирует содержимое кода и качество ревью, не проверяет всю аренду, остановку или сеть. Отключения показывают необходимость правил для данного свойства, но не дают оценку пропуска дефектов моделями. Следующий шаг — механически проверяемая спецификация более широкого протокола [10]. Работающие срезы рантайма перечислены в таблице раздела «Swarm Runtime»; они не являются подтверждением распределённого слоя.

Проверка соблюдения протокола должна использовать программных нарушителей: выход за область записи, просроченную аренду, подмену кандидата, изменение критериев после ревью, повтор доставки и потерю исхода операции. Ожидаемый результат — отсутствие запрещённого эффекта и восстановление по журналу. Отдельный опыт удаляет сессию оркестратора: преемник должен продолжить по записям и артефактам, не по скрытой памяти модели. Эти опыты описаны, но результатов реализации здесь нет. ⟦предложено⟧

Полезность проверяется отдельно: одиночный агент, параллельные агенты одного владельца с общим репозиторием и блокировками, развёртывание L0. Одинаковы

внешние задачи, критерии принятия и ресурсные ограничения. В стоимость входят inference, проверки, занятость среды и вмешательство человека. Измеряются ожидание слотов и областей, повторная работа после сдвига головы, восстановление, пересмотренные отклонения и откаты. Качество ревью предлагается калибровать посевом дефектов; такой опыт оценивает обнаружение выбранных классов, но не долю злоумышленников и не независимость совместных ошибок. Для памяти нужен отдельный пилот против обычного поиска с учётом цены поддержания корпуса. ⟦предложено⟧

9. Чего платформа не обещает

Независимое ревью не доказывает правильности суждения. Несколько моделей с одинаковыми слепыми зонами могут согласованно пропустить дефект. Подпись удостоверяет автора вердикта; кворум удостоверяет соблюдение правил при допущениях уровня, а не истинность оценки.

Платформа не предотвращает компрометацию самой среды или ошибку ядра. Подписанный receipt скомпрометированной среды не становится фактом: обнаружение требует независимого наблюдения. На L1 злонамеренный секвенсер может раздвоить историю; при недоступном кворуме или подходящих ревьюверах продвижение не гарантируется. Возможность остановки намеренно позволяет потерять живучесть ради сдерживания.

Код санбокса виден контрибьютору и провайдеру его модели. Ограничение egress не обещает скрыть уже прочитанное и не отзывает доставленные модели байты. Подтверждение идентичности не доказывает честности владельца, репутация не исключает сговор, а предложенная экономика не имеет доказанной справедливости. Цепочка не обещает вечного хранения контента. Эти границы должны оставаться видимыми в политике проекта, а не исчезать за словом «доверие».

10. Смежные работы

Prove2Me объединяет агентов независимых владельцев для формализации математики [7]; пример совместной формализации приведён в [8]. Общность — вклад чужой ёмкости и разделение формулировки задачи и результата. Различие — ядро доказательств как оракул принятия. Опыт параллельных агентов при создании компилятора [9] ближе к разработке, но не задаёт той же границы между недоверенными владельцами.

BFT-протоколы задают условия целостности распределённого журнала [1, 2], а BOINC — образец адаптивной репликации по истории участника [3]. Capability-подход даёт модель ограниченного делегирования; в источниках рантайма рассматриваются SPKI/SDSI, Macaroons, UCAN и Biscuit, но выбор реализации не закреплён. Пределы репутации и устойчивости к множественным личностям обсуждаются в [5, 4].

Memo служит образцом социального словаря и связей ключей, Namecoin — реестра имён [11, 12, 13]. Nostr, Bluesky и Farcaster дают материал для сравнения идентичности, распространения и модерации: вынесение данных из цепочки не устраняет риск централизации индекса и клиента. Для Swarm Chain поэтому предлагается канонический контракт индексатора; федерация транспорта сама по себе его не заме-

няет [18, 19, 20]. Ни один из этих примеров не подтверждает работоспособность всей композиции Swarma.

11. Заключение

Swarma связывает три задачи: ограничить агента на машине владельца, допустить чужую ёмкость к совместной разработке и дать личности историю вне одного проекта. Общие принципы — сужение полномочий, авторитетные записи, точная адресация одобрений и отделение суждения от детерминированной проверки. Степень готовности различна: отдельные компоненты работают, распределённый протокол спроектирован, цепочка и переносимая репутация предложены.

Перебор подтверждает ограниченный фрагмент интеграции и обнаруживает нарушения при снятии каждого правила. Следующее свидетельство должно прийти от реализации с программными нарушителями и сравнения на внешних задачах. Если расходы на координацию и повторную проверку превышают пользу, гипотеза об эффективности роя не подтверждается. Границы доверия полезны только тогда, когда их можно проверить и когда цена их соблюдения измерена.

Список литературы

[1] M. Castro, B. Liskov. Practical Byzantine Fault Tolerance. OSDI, 1999.
[2] M. Yin et al. HotStuff: BFT Consensus with Linearity and Responsiveness. PODC, 2019.
[3] D. P. Anderson. BOINC: A System for Public-Resource Computing and Storage. IEEE/ACM International Workshop on Grid Computing, 2004.
[4] A. Cheng, E. Friedman. Sybilproof Reputation Mechanisms. ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems, 2005.
[5] J. R. Douceur. The Sybil Attack. IPTPS, 2002.
[6] S. Micali, M. Rabin, S. Vadhan. Verifiable Random Functions. FOCS, 1999.
[7] S. Chen et al. Prove2Me: An Open Collaborative Platform for Scaling Math Formalization. arXiv:2608.28433, 2026.
[8] Anthropic. Formalizing Fermat's Last Theorem. anthropic.com/research, 2026.
[9] N. Carlini. Building a C Compiler with a Team of Parallel Claudes. Anthropic Engineering, 2026.
[10] L. Lamport. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.
[11] Memo. Protocol.
[12] Memo. Identity.
[13] Namecoin. FAQ.
[14] An Empirical Study of Namecoin and Lessons for Decentralized Namespace Design. 2015.
[15] Bitcoin Cash Node. ASERT specification.
[16] RandomX. Design.
[17] Monero. Tail emission.
[18] Nostr. NIP-32: Labeling.
[19] Bluesky. Stackable moderation.
[20] Neynar. Neynar is acquiring Farcaster.